Technology Stack

Date	02 july 2026
Team ID	SWTID-2026-2662
Project Name	Ai Powered Debt Relief & Financial Recovery Platform
Maximum Marks	2 Marks

Define Your Technology Stack:

Identify the programming languages, frameworks, databases, front-end tools, back-end tools, and APIs that you will use to build your solution. A well-chosen technology stack ensures that your application is scalable, maintainable, and aligned with your project requirements.

Fill out the table below to detail the specific technologies chosen for each component of your architecture and provide a brief justification for your choice.

Technology Stack Details Table

S.No	Architecture Component/Layer	Technology Chosen	Justification / Purpose
1	**Frontend / Client-Side**	**Streamlit (v1.35.0)** with **Custom HTML/CSS** & **Google Fonts** (*Syne, DM Sans, JetBrains Mono*)	Chosen for rapid frontend prototyping completely within the Python ecosystem, removing JavaScript overhead. It provides reactive widgets (sliders, tabs, expanders) out-of-the-box. Custom CSS was injected to implement a premium, dark-mode visual system featuring custom dashboard metric cards, stress progress bars, and dedicated, scrollable text blocks to make the financial data easily digestible for stressed users.
2	**Backend / Server-Side**	**FastAPI (v0.111.0)** run via **Uvicorn (v0.29.0)**	Selected for its exceptional performance, native asynchronous (`async/await`) capabilities, and speed. FastAPI auto-generates interactive Swagger API documentation (`/docs`) for debugging backend controllers. It utilizes **Pydantic (v2.7.1)** for data-

			contract validation and enforces secure JWT-based authorization boundaries.
3	**Database / Data Storage**	**SQLite** with **SQLAlchemy ORM (v2.0.30)**	SQLite provides a zero-configuration, self-contained relational database that stores financial models directly in a single `finrelief.db` file. SQLAlchemy ORM decouples SQL query execution, simplifies relationships (User ↔ Loan ↔ Settlement mapping), prevents SQL injection, and enforces relational integrity via cascade delete-orphan configurations.
4	**Cloud / Hosting / Deployment**	**Docker** & **Render** (or **AWS EC2 / Elastic Beanstalk**)	Containerizing the Streamlit frontend and FastAPI backend into Docker images guarantees identical execution across local and production environments. Hosting on modern platforms like Render allows independent, cost-effective deployments, automated builds from git triggers, and auto-scaling support for peak borrower traffic.
5	**Version Control & CI/CD**	**Git** & **GitHub** with **GitHub Actions**	Git manages step-by-step codebase progression, code branching, and rollback paths. GitHub hosts the remote repository to streamline developer collaboration, while GitHub Actions provides automated CI/CD pipelines to run test validations, check syntax compliance, and auto-deploy to production on target merges.
6	**Third-Party APIs / Other Tools**	**Google Gemini API** (via `google-generativeai` SDK v0.5.4), **PyJWT (`python-jose` v3.3.0)** & **Werkzeug Security**	The **Gemini 1.5 Flash** model dynamically drafts highly personalized lender negotiation letters based on pre-calculated financial stress margins. **Werkzeug** securely hashes passwords to prevent raw storage, while **python-jose** issues and validates stateless JWT tokens to protect borrower profiles from unauthorized access.